

```
In [1]: import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler, Multi
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.multiclass import OneVsRestClassifier
from sklearn.metrics import accuracy_score

import xgboost as xgb
df = pd.read_csv("../data/filteredvehpub.csv") # adjust path as need
df.head()
df.shape

## Read file
```

```
Out[1]: (256115, 14)
```

```
In [2]: df.columns
```

```
Out[2]: Index(['HOUSEID', 'HHFAMINC', 'HHSIZE', 'LIF_CYC', 'CENSUS_R', 'URBA
N',
              'VEHID', 'DRVRCNT', 'MAKE', 'HH_RACE', 'WRKCOUNT', 'HOMEOWN',
              'URBANSIZE', 'VEHYEAR'],
              dtype='object')
```

```
In [3]: import numpy as np
invalid_values = [-9, -8, -7, -88, 99, "99", "XX", "xx", "XX ", "-88",
df = df.replace(invalid_values, np.nan)
df = df.dropna()

## Clean data
```

```
In [4]: #df["VEHTYPE"] = df["VEHTYPE"].replace([5, 6], np.nan)
#df = df.dropna()
```

```
In [5]: counts = df["MAKE"].value_counts()
counts.describe()
```

```
Out[5]: count      53.000000
mean      4607.245283
std       8163.852477
min        58.000000
25%       286.000000
50%      1580.000000
75%      4639.000000
max      34768.000000
Name: count, dtype: float64
```

```
In [6]: df["MAKE"] = df["MAKE"].astype(str)    # convert everything to string f
df["MAKE"] = df["MAKE"].str.strip()          # remove whitespace
df["MAKE"] = df["MAKE"].astype(int)          # convert to integer
## Did this because the original data has different variable types, wa

counts = df["MAKE"].value_counts()
rare_makes = counts[counts < 2000].index
## Changed to 2000 for better accuracy

df["MAKE"] = df["MAKE"].where(~df["MAKE"].isin(rare_makes), 98)
df["MAKE"].value_counts()
## Put them all into to other
```

```
Out[6]: MAKE
12      34768
49      33623
20      31188
37      23829
98      21456
7       12545
35      11684
2        7293
48       6932
23       6896
55       6765
59       4967
18       4852
6        4703
63       4639
30       4608
34       4536
41       4335
42       3642
72       3064
19       2664
22       2659
54       2536
Name: count, dtype: int64
```

```
In [7]: #df.to_csv("processed_data.csv", index=False)
```

```
In [8]: hh_makes = (
    df.groupby("HOUSEID")["MAKE"]
      .apply(lambda s: sorted(set(s))) # unique list of makes
      .reset_index(name="MAKE_LIST")
)

hh_makes.head()

## Group vehicles into one list for each household
```

Out [8]:

	HOUSEID	MAKE_LIST
0	30000007	[19, 20, 49]
1	30000008	[20]
2	30000012	[12, 98]
3	30000019	[37, 98]
4	30000029	[20, 49]

```
In [9]: ## Selecting Household-Level Features

household_feature_cols = [
    "HOUSEID",
    "HHSIZE",
    "HHFAMINC",
    "LIF_CYC",
    "CENSUS_R",
    "HH_RACE",
    "HOMEOWN",
    "WRKCOUNT",
    "URBAN",
    "URBANSIZE",
    "DRVRCNT"
]

## Ensuring One Row per Household

hh_feat = df[household_feature_cols].drop_duplicates("HOUSEID")

# Merge features with MAKE_LIST
hh = hh_feat.merge(hh_makes, on="HOUSEID")
hh.head()
```

Out [9]:

	HOUSEID	HHSIZE	HHFAMINC	LIF_CYC	CENSUS_R	HH_RACE	HOMEOWN
0	30000007	3	7.0	10.0	3	2.0	1.0
1	30000008	2	8.0	2.0	2	1.0	1.0
2	30000012	1	10.0	1.0	1	1.0	1.0
3	30000019	2	3.0	2.0	3	1.0	1.0
4	30000029	2	5.0	10.0	2	1.0	1.0

In []:

```
In [10]: ## features & targets

feature_cols = [
    "HHSIZE",
```

```

    "HHFAMINC",
    "LIF_CYC",
    "CENSUS_R",
    "HH_RACE",
    "HOMEOWN",
    "WRKCOUNT",
    "URBAN",
    "URBANSIZE",
    "DRVRCNT"
]

X = hh[feature_cols]
y_list = hh["MAKE_LIST"] # list of makes per household

# Multi-label binarization: list of makes → multi-hot matrix
mlb = MultiLabelBinarizer()
Y = mlb.fit_transform(y_list)

print("Classes (makes):", mlb.classes_)
print("Y shape:", Y.shape) # (n_households, n_makes)]

```

```

Classes (makes): [ 2  6  7 12 18 19 20 22 23 30 34 35 37 41 42 48 49 54
55 59 63 72 98]
Y shape: (118549, 23)

```

In [11]: # train test separate

```

X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y,
    test_size=0.2,
    random_state=42
)

```

In [12]: ## Separate numeric features with categorical features
Then use standardScaler() for num and One-hot encoding for cat

```

numeric_features = ["HHSIZE", "HHFAMINC", "WRKCOUNT", "DRVRCNT"]
categorical_features = ["LIF_CYC", "CENSUS_R", "HH_RACE", "HOMEOWN", "

preprocess = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_fe
    ]
)

```

In [13]: base_xgb = xgb.XGBClassifier(
 n_estimators=300, # number of boosted trees
 max_depth=5, # tree depth (controls model complexity)
 learning_rate=0.1, # shrinkage rate for boosting updates
 subsample=0.8, # % of rows sampled per tree (prevents overfitting)
 colsample_bytree=0.8, # % of features sampled per tree

```

    objective="binary:logistic", # 0vR: each label is treated as bina
    eval_metric="logloss",      # evaluation metric for training
    tree_method="hist",         # fast histogram-based tree building
    n_jobs=-1,                  # use all CPU cores
    random_state=42              # reproducibility
)

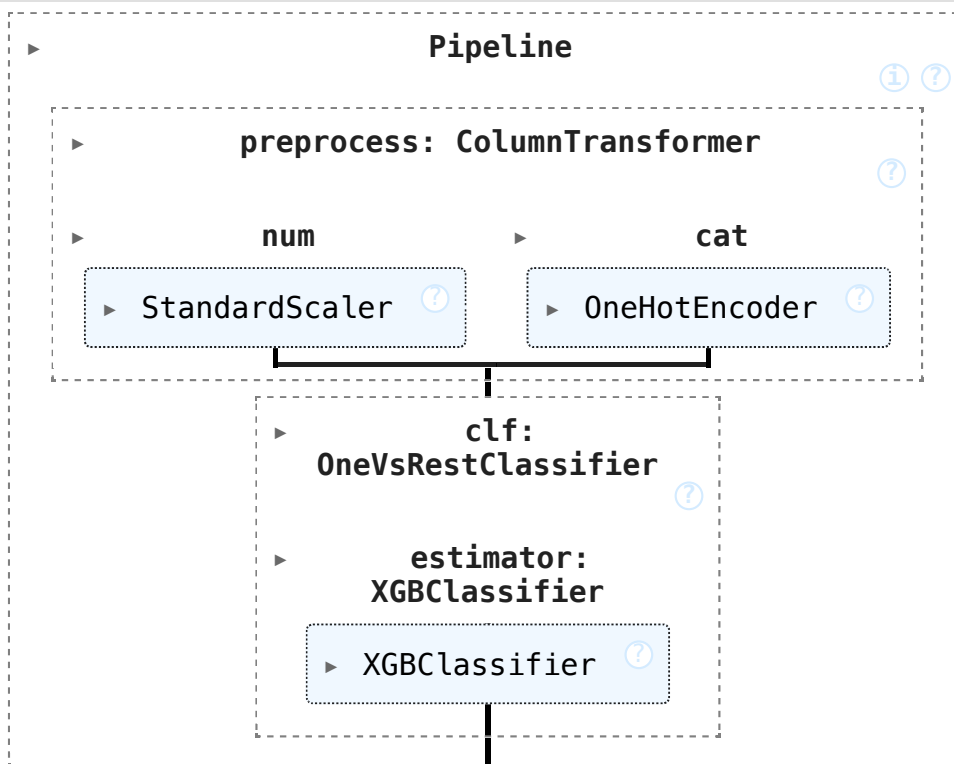
multi_xgb = OneVsRestClassifier(base_xgb, n_jobs=-1) # wrap into mult

pipe = Pipeline([
    ("preprocess", preprocess), # scaling + one-hot encoding
    ("clf", multi_xgb)          # multi-label XGBoost classifier
])

```

In [14]: `pipe.fit(X_train, Y_train)`

Out[14]:



```

In [15]: # Access fitted classifier and transformed test data
clf_step = pipe.named_steps["clf"]
X_test_trans = pipe.named_steps["preprocess"].transform(X_test)

# Get probability that each household has each make
probs_per_class = np.column_stack([
    est.predict_proba(X_test_trans)[:, 1] # P(has this make)
    for est in clf_step.estimators_
])

probs_per_class.shape # (n_samples, n_classes)

```

Out[15]: (23710, 23)

```
In [16]: top1_idx = np.argmax(probs_per_class, axis=1) # best make index per
top1_makes = mlb.classes_[top1_idx] # corresponding make n

top1_makes[:10]
```

```
Out[16]: array([98, 12, 20, 20, 49, 37, 49, 49, 37, 20])
```

```
In [17]: correct_flags = []

for i in range(Y_test.shape[0]):
    # Y_test[i, top1_idx[i]] == 1 if household i actually owns that pr
    correct_flags.append(Y_test[i, top1_idx[i]] == 1)

top1_in_set_accuracy = np.mean(correct_flags)
print("XGBoost Top-1-in-set accuracy:", top1_in_set_accuracy)
```

XGBoost Top-1-in-set accuracy: 0.2858709405314213

```
In [18]: ## Dummy Classifier

# Find most common make in the entire dataset
all_makes_flat = [m for makes in y_list for m in makes]
most_common_make = pd.Series(all_makes_flat).value_counts().idxmax()
print("Most common make:", most_common_make)

# Find its index in mlb.classes_
baseline_idx = np.where(mlb.classes_ == most_common_make)[0][0]

# Compute baseline Top-1-in-set accuracy
baseline_flags = []
for i in range(Y_test.shape[0]):
    baseline_flags.append(Y_test[i, baseline_idx] == 1)

baseline_top1_in_set = np.mean(baseline_flags)
print("Baseline Top-1-in-set accuracy (always predict most common make)
print("XGBoost Top-1-in-set accuracy:", top1_in_set_accuracy)
```

Most common make: 12

Baseline Top-1-in-set accuracy (always predict most common make): 0.233
90974272458878

XGBoost Top-1-in-set accuracy: 0.2858709405314213

```
In [19]: # Get the newest vehicle per household for the confusion matrix. Check
newest_vehicle = df.sort_values(["HOUSEID", "VEHYEAR"], ascending=[True
primary_make_df = newest_vehicle.drop_duplicates("HOUSEID")["HOUSEID"]
primary_make_df.rename(columns={"MAKE": "PRIMARY_MAKE"}, inplace=True)

# Build MAKE_LIST
hh_makes = (
    df.groupby("HOUSEID")["MAKE"]
    .apply(lambda s: sorted(set(s)))
    .reset_index(name="MAKE_LIST")
)
```

```
# Merge to get PRIMARY_MAKE + MAKE_LIST per household
hh = hh_feat.merge(hh_makes, on="HOUSEID").merge(primary_make_df, on="
```

```
In [20]: primary_labels = hh["PRIMARY_MAKE"].values
X = hh[feature_cols]
Y_multilabel = mlb.fit_transform(hh["MAKE_LIST"])
Y_primary = primary_labels # for confusion matrix
X_train, X_test, Y_ml_train, Y_ml_test, Y_pr_train, Y_pr_test = train_
    X, Y_multilabel, Y_primary,
    test_size=0.2,
    random_state=42
)
```

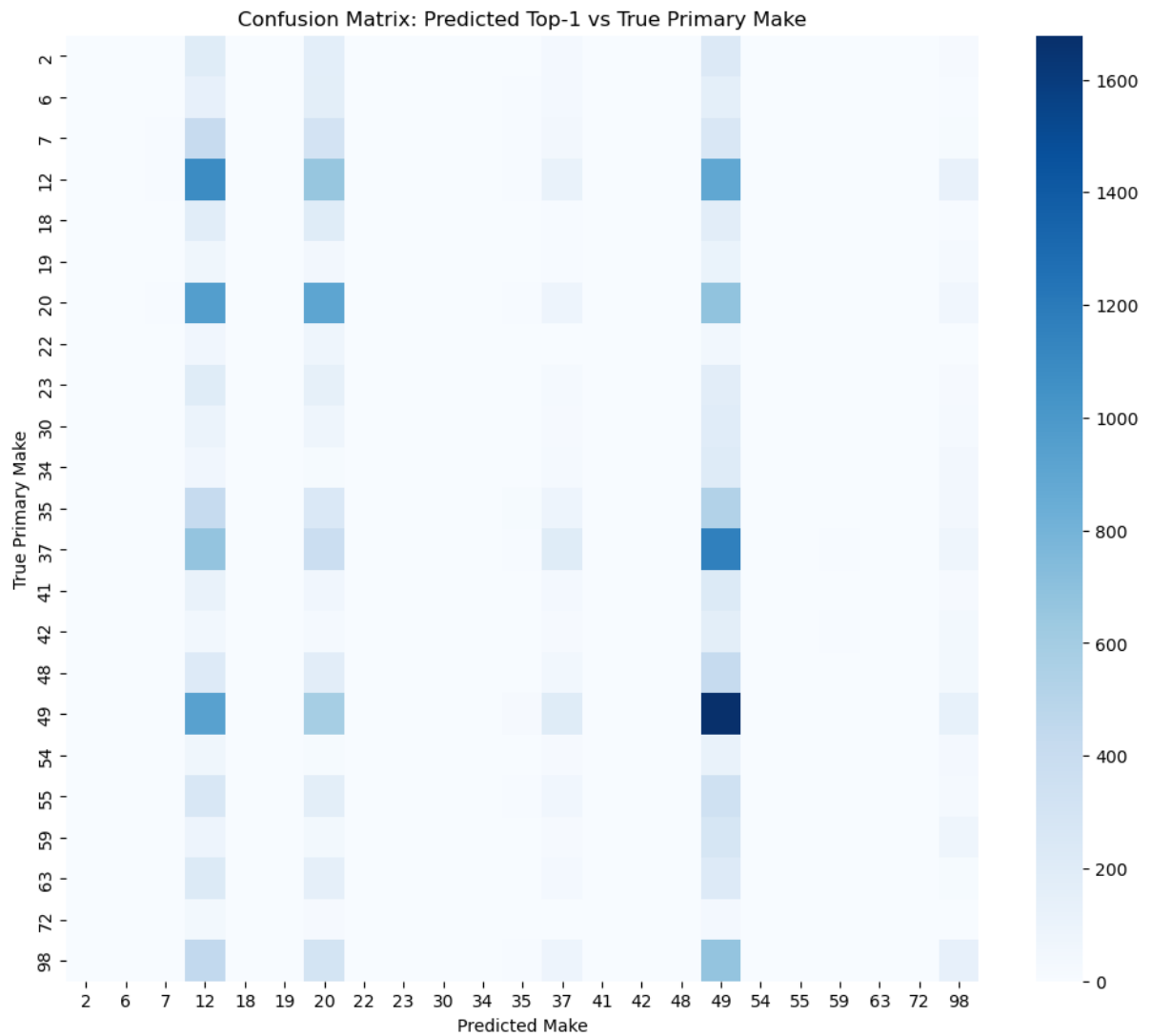
```
In [ ]:
```

```
In [21]: ## Confusion Matrix

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(Y_pr_test, top1_makes, labels=mlb.classes_)

plt.figure(figsize=(12, 10))
sns.heatmap(
    cm,
    cmap='Blues',
    xticklabels=mlb.classes_,
    yticklabels=mlb.classes_
)
plt.xlabel("Predicted Make")
plt.ylabel("True Primary Make")
plt.title("Confusion Matrix: Predicted Top-1 vs True Primary Make")
plt.show()
```



In []:

In []:

```
In [22]: feature_cols = [
    "HHSIZE",
    "HHFAMINC",
    "LIF_CYC",
    "CENSUS_R",
    "HH_RACE",
    "HOMEOWN",
    "WRKCOUNT",
    "URBAN",
    "URBANSIZE",
    "DRVRCNT"
]
```

```
In [23]: ## Create a GUI for the recommendation function
```

```
import numpy as np
```



```

import pandas as pd
import ipywidgets as widgets
from IPython.display import display, clear_output

# ----- 1. Helper: recommendation function -----

def recommend_makes(user_features, pipe, mlb, top_k=5):
    """
    user_features: dict with keys matching feature_cols
    pipe: trained sklearn Pipeline (preprocess + OneVsRest(XGB/RF))
    mlb: fitted MultiLabelBinarizer
    top_k: how many makes to return
    """
    X_new = pd.DataFrame([user_features])

    preprocess = pipe.named_steps["preprocess"]
    clf = pipe.named_steps["clf"]

    X_new_trans = preprocess.transform(X_new)

    # P(has this make) for each class
    probs = np.column_stack([
        est.predict_proba(X_new_trans)[: , 1]
        for est in clf.estimators_
   ])[0] # shape: (n_classes,)

    top_k = min(top_k, len(mlb.classes_))
    top_idx = np.argsort(probs)[: -1][ : top_k]

    top_makes = mlb.classes_[top_idx]
    top_probs = probs[top_idx]

    return list(zip(top_makes, top_probs))

# ----- 2. Build dropdowns from data (with human-readable labels)

# These must match training feature columns
feature_cols = [
    "HHSIZE",
    "HHFAMINC",
    "LIF_CYC",
    "CENSUS_R",
    "HH_RACE",
    "HOMEOWN",
    "WRKCOUNT",
    "URBAN",
    "URBANSIZE",
    "DRVRCNT"
]

# Label maps

```

```
hhfaminc_label_map = {
    1: "Less than $10,000",
    2: "$10,000-$14,999",
    3: "$15,000-$24,999",
    4: "$25,000-$34,999",
    5: "$35,000-$49,999",
    6: "$50,000-$74,999",
    7: "$75,000-$99,999",
    8: "$100,000-$124,999",
    9: "$125,000-$149,999",
    10: "$150,000-$199,999",
    11: "$200,000 or more",
}

lif_cyc_label_map = {
    1: "1 adult, no children",
    2: "2+ adults, no children",
    3: "1 adult, youngest child 0-5",
    4: "2+ adults, youngest child 0-5",
    5: "1 adult, youngest child 6-15",
    6: "2+ adults, youngest child 6-15",
    7: "1 adult, youngest child 16-21",
    8: "2+ adults, youngest child 16-21",
    9: "1 adult, retired, no children",
    10: "2+ adults, retired, no children",
    -9: "Not ascertained",
}

census_r_label_map = {
    1: "Northeast",
    2: "Midwest",
    3: "South",
    4: "West",
}

hh_race_label_map = {
    1: "White",
    2: "Black or African American",
    3: "Asian",
    4: "American Indian / Alaska Native",
    5: "Native Hawaiian / Pacific Islander",
    6: "Multiple races",
    7: "Other race",
    8: "Refused",
    97: "Don't know",
}

homeown_label_map = {
    1: "Own",
    2: "Rent",
}
```

```
    97: "Some other arrangements",
}

urbansize_label_map = {
    1: "Urban 50k-199,999",
    2: "Urban 200k-499,999",
    3: "Urban 500k-999,999",
    4: "Urban ≥1M (no heavy rail)",
    5: "Urban ≥1M (with heavy rail)",
    6: "Non-urban / other",
}

urban_label_map = {
    1: "In an urban area",
    2: "In an urban cluster",
    3: "In an area surrounded by urban areas",
    4: "Not in urban area",
}

make_label_map = {
    1: "American Motors",
    2: "Jeep",
    3: "AM General",
    6: "Chrysler/Daimler Chrysler",
    7: "Dodge",
    9: "Plymouth",
    12: "Ford",
    13: "Lincoln",
    14: "Mercury",
    18: "Buick",
    19: "Cadillac",
    20: "Chevrolet",
    21: "Oldsmobile",
    22: "Pontiac",
    23: "GMC",
    24: "Saturn",
    29: "Other Domestic",
    30: "Volkswagen",
    32: "Audi",
    34: "BMW",
    35: "Nissan/Datsun",
    36: "Fiat",
    37: "Honda",
    38: "Isuzu",
    39: "Jaguar",
    41: "Mazda",
    42: "Mercedes-Benz",
    43: "MG",
    45: "Porsche",
    47: "Saab",
    48: "Subaru",
```

```

49: "Toyota",
50: "Triumph",
51: "Volvo",
52: "Mitsubishi",
53: "Suzuki",
54: "Acura",
55: "Hyundai",
58: "Infiniti",
59: "Lexus",
62: "Land Rover",
63: "Kia",
65: "Smart",
67: "Scion",
69: "Other Import",
82: "Freightliner",
84: "International Harvester/Navistar",
98: "Other",
99: "Unknown",
}

feature_label_maps = {
    "HHFAMINC": hhfaminc_label_map,
    "LIF_CYC": lif_cyc_label_map,
    "CENSUS_R": census_r_label_map,
    "HH_RACE": hh_race_label_map,
    "HOMEOWN": homeown_label_map,
    "URBAN": urban_label_map,
    "URBANSIZE": urbansize_label_map,
    # HHSIZE, WRKCOUNT, DRVRCNT we already show as plain ints
}

def decode_make(val):
    # Map a MAKE code to a human-readable brand name.
    # try both int and str versions to be safe
    if val in make_label_map:
        return make_label_map[val]
    try:
        return make_label_map[int(val)]
    except Exception:
        return str(val)

# Helper: build dropdown with (nice_label, raw_code) options
def coded_dropdown(df, col, label_map, description):
    vals = df[col].dropna().unique()
    try:
        vals = sorted(vals)
    except TypeError:
        vals = sorted(vals, key=lambda x: str(x))
    options = []
    for v in vals:
        label = label_map.get(v, label_map.get(str(v), str(v)))
        options.append((label, v)) # (what user sees, actual value)

```

```

    return widgets.Dropdown(
        options=options,
        description=description,
        style={'description_width': 'initial'}
    )

# HHSIZE – build readable "N persons"
hhs_vals = sorted(df["HHSIZE"].dropna().unique())
hhs_options = [
    (f"{int(v)} person" if int(v) == 1 else f"{int(v)} persons", v)
    for v in hhs_vals
]
hhs_widget = widgets.Dropdown(
    options=hhs_options,
    description="Household size:",
    style={'description_width': 'initial'}
)

# Income category
inc_widget = coded_dropdown(df, "HHFAMINC", hhfaminc_label_map, "House

# Life cycle
lif_widget = coded_dropdown(df, "LIF_CYC", lif_cyc_label_map, "Life cy

# Region
reg_widget = coded_dropdown(df, "CENSUS_R", census_r_label_map, "Censu

# Race
race_widget = coded_dropdown(df, "HH_RACE", hh_race_label_map, "Race:"

# Home ownership
homeown_widget = coded_dropdown(df, "HOMEOWN", homeown_label_map, "Hom

# # Workers – readable labels
wrk_vals = sorted(df["WRKCOUNT"].dropna().unique())
wrk_options = [
    (f"{int(v)} worker" if int(v) == 1 else f"{int(v)} workers", v)
    for v in wrk_vals
]
wrk_widget = widgets.Dropdown(
    options=wrk_options,
    description="# Workers:",
    style={'description_width': 'initial'}
)

# Urban / rural flag
urban_widget = coded_dropdown(df, "URBAN", urban_label_map, "Urban / r

# Urban size
urbsize_widget = coded_dropdown(df, "URBANSIZE", urbansize_label_map,

# # Drivers – readable labels

```

```

drv_vals = sorted(df["DRVRCNT"].dropna().unique())
drv_options = [
    (f"{int(v)} driver" if int(v) == 1 else f"{int(v)} drivers", v)
    for v in drv_vals
]
drv_widget = widgets.Dropdown(
    options=drv_options,
    description="# Drivers:",
    style={'description_width': 'initial'}
)

topk_widget = widgets.IntSlider(
    value=5,
    min=1,
    max=10,
    step=1,
    description="Top-K:",
)

run_button = widgets.Button(
    description="Recommend Makes",
    button_style="success"
)

output_area = widgets.Output()

# ----- 3. Callback: when button is clicked -----

def on_run_clicked(b):
    with output_area:
        clear_output()

        # Collect user-selected values (note: .value is the raw code)
        user_features = {
            "HHSIZE": hhs_widget.value,
            "HHFAMINC": inc_widget.value,
            "LIF_CYC": lif_widget.value,
            "CENSUS_R": reg_widget.value,
            "HH_RACE": race_widget.value,
            "HOMEOWN": homeown_widget.value,
            "WRKCOUNT": wrk_widget.value,
            "URBAN": urban_widget.value,
            "URBANSIZE": urbsize_widget.value,
            "DRVRCNT": drv_widget.value,
        }

        # Get recommendations
        top_k = topk_widget.value
        recs = recommend_makes(user_features, pipe, mlb, top_k=top_k)

        print("Input household profile:")

```

```

    for k, v in user_features.items():
        label_map = feature_label_maps.get(k)
        if label_map is not None:
            # Try int and str keys
            human = label_map.get(v, label_map.get(str(v), v))
        else:
            human = v
        print(f" {k}: {human} (code={v})")

    # ---- Pretty-print recommended makes using brand names ----
    print(f"\nRecommended makes (Top-{top_k}):")
    for make_code, p in recs:
        make_name = decode_make(make_code)
        print(f" {make_name:<20} (code={make_code})   prob ≈ {p:.

run_button.on_click(on_run_clicked)

# ----- 4. Display the GUI -----

ui_left = widgets.VBox([
    hhs_widget,
    inc_widget,
    lif_widget,
    reg_widget,
    race_widget,
])

ui_right = widgets.VBox([
    homeown_widget,
    wrk_widget,
    urban_widget,
    urbsize_widget,
    drv_widget,
    topk_widget,
    run_button,
])

ui = widgets.HBox([ui_left, ui_right])

display(ui, output_area)

```

```

HBox(children=(VBox(children=(Dropdown(description='Household size:', o
ptions=(('1 person', 1), ('2 persons', ...
Output()

```

In []:

In []:

In [24]: *## Get important features from XGB model*
import numpy **as** np

```
import pandas as pd

# 1. Get feature names after preprocessing
preprocess = pipe.named_steps["preprocess"]
feature_names = preprocess.get_feature_names_out()
print("Number of transformed features:", len(feature_names))
```

Number of transformed features: 38

```
In [25]: # 2. Get estimators from OneVsRestClassifier
estimators = pipe.named_steps["clf"].estimators_

# 3. Stack feature_importances_ from each XGB model
all_importances = np.vstack([
    est.feature_importances_ for est in estimators
]) # shape: (n_classes, n_features)

print("Importances shape:", all_importances.shape)

# 4. Average importance across all classes (makes)
mean_importance = all_importances.mean(axis=0) # shape: (n_features,)
print("Mean importance length:", len(mean_importance))
print("Feature names length:", len(feature_names))
```

Importances shape: (23, 38)

Mean importance length: 38

Feature names length: 38

```
In [26]: # 5. Build DataFrame
importance_df = pd.DataFrame({
    "feature": feature_names,
    "importance": mean_importance
}).sort_values("importance", ascending=False)

importance_df.head(15)
```


Out [26]:

	feature	importance
28	cat__URBAN_1	0.056783
20	cat__HH_RACE_3.0	0.053421
37	cat__URBANSIZE_6	0.047656
15	cat__CENSUS_R_2	0.042282
3	num__DRVRCNT	0.041467
1	num__HHFAMINC	0.036144
13	cat__LIF_CYC_10.0	0.031953
36	cat__URBANSIZE_5	0.031174
14	cat__CENSUS_R_1	0.030556
17	cat__CENSUS_R_4	0.030266
19	cat__HH_RACE_2.0	0.028586
16	cat__CENSUS_R_3	0.027977
31	cat__URBAN_4	0.027087
2	num__WRKCOUNT	0.026341
26	cat__HOMEOWN_2.0	0.026318

In []: